

ĐỀ BÀI KIỂM TRA NĂNG LỰC: ANALYTICS ENGINEER (Thời gian: 48h)

Bối cảnh (Business Case): Bạn đang đảm nhận vai trò Analytics Engineer cho một chuỗi bán lẻ điện thoại di động với hơn 170 cửa hàng. Công ty đang gặp vấn đề về việc dữ liệu tồn kho, doanh số và lưu lượng khách (traffic) bị phân mảnh ở nhiều hệ thống khác nhau, dẫn đến báo cáo chậm trễ và thiếu tính nhất quán.

Phần 1: Data Modeling (25 điểm)

Đầu vào: Hệ thống ghi nhận 3 luồng dữ liệu thô:

1. Transactions: Giao dịch bán hàng. (Transaction_ID, Date, Product_ID, Store_ID, Quantity, Unit_Price).
2. Inventory_Logs: Lịch sử xuất/nhập/tồn kho hằng ngày tại từng cửa hàng. (Log_Date, Store_ID, Product_ID, Beginning_Inventory, Received, Sold, Ending_Inventory)
3. Store_Info: Thông tin cửa hàng (Store_ID, Store_Name, Region, RSM, AM, Store_Type).

Yêu cầu:

- Hãy thiết kế một mô hình dữ liệu (Data Warehouse Schema - Star hoặc Snowflake) tối ưu cho nền tảng BigQuery để phục vụ việc tạo báo cáo BI hiệu quả.
- Trình bày sơ đồ thực thể liên kết (ERD) mô tả rõ các bảng Fact và Dimension.
- **Câu hỏi tư duy:** Khi giá bán của một mẫu điện thoại thay đổi liên tục theo các chương trình khuyến mãi, bạn sẽ áp dụng kỹ thuật Slowly Changing Dimension (SCD) loại mấy cho bảng Dimension Sản phẩm và tại sao?

Phần 2: Xử lý dữ liệu bằng Python (25 điểm)

Đầu vào: Các tệp dữ liệu giả lập .csv chứa được đính kèm bài test.

Yêu cầu: Viết một đoạn script bằng Python (sử dụng Pandas, PySpark hoặc thư viện phù hợp) để thực hiện Data Pipeline cơ bản:

- **Làm sạch (Clean):** Xử lý các giá trị null, loại bỏ dữ liệu giao dịch trùng lặp và chuẩn hóa định dạng thời gian.
- **Biến đổi (Transform):** Viết logic tính toán thêm cột Daily_Run_Rate (tốc độ bán hàng trung bình ngày) và gộp dữ liệu tồn kho với dữ liệu bán hàng để tính tỷ lệ Tồn kho/Doanh số (Inventory to Sales Ratio).
- Đầu ra là một tệp dữ liệu sạch, đã được cấu trúc hóa sẵn sàng đẩy lên Data Warehouse.
- *Lưu ý: Ứng viên cần nộp lại file .py hoặc Jupyter Notebook, chú trọng vào việc viết code rõ ràng, dễ bảo trì.*

Phần 3: Trực quan hóa & Phân tích bằng Power BI (30 điểm)

Yêu cầu: Sử dụng tập dữ liệu đầu ra từ Phần 2 để xây dựng một Dashboard vận hành trên Power BI đáp ứng các tiêu chí:

- **Thiết kế Dashboard:** Thể hiện tổng quan tình hình kinh doanh của các khu vực, làm nổi bật được các cửa hàng đang có hiệu suất cao và cửa hàng có cảnh báo về tồn kho (thiếu hàng hoặc tồn đọng quá lâu).
- **Kỹ năng DAX nâng cao:** Viết các DAX measure phức tạp để tính toán:
 1. Tỷ lệ hoàn thành mục tiêu (Achievement %) sử dụng bảng ảo (Virtual Tables).
 2. Phân bổ doanh thu theo loại cửa hàng.
- Đánh giá dựa trên tư duy UI/UX, cách sắp xếp semantic model tối ưu và logic của DAX.

Phần 4: Tư duy Hệ thống & Quản trị Codebase (20 điểm)

Tình huống 1 (Tối ưu Hệ thống - 10 điểm): Pipeline dữ liệu tự động hằng ngày trên BigQuery đột ngột tăng vọt chi phí (cost) gấp 3 lần và chạy rất chậm, dù lượng dữ liệu đầu vào không có sự đột biến.

- **Yêu cầu:** Gạch đầu dòng ngắn gọn các bước bạn sẽ thực hiện để điều tra (troubleshoot) và các kỹ thuật tối ưu SQL sẽ áp dụng để đưa hệ thống ổn định trở lại.

Tình huống 2 (Quản trị Logic & Codebase - 10 điểm): Trong quá trình rà soát một Data Pipeline khác, bạn thấy một đoạn script Python dùng để phân loại cửa hàng (shop classification) đang được team sử dụng:

```
def classify_shop(score, kpi_count):  
    if score >= 1 and kpi_count >= 6:  
        return "Xuất sắc"  
    if score >= 0.85 and kpi_count >= 5:  
        return "Tốt"  
    if score >= 0.7 and kpi_count >= 4:  
        return "Khá"  
    if score >= 0.5 and kpi_count >= 2:  
        return "Kém"  
    return "Cảnh báo"
```

Team Business báo cáo rằng số liệu phân loại cửa hàng tháng này đang bị mâu thuẫn (inconsistent) với số liệu của các tháng trước. Nguyên nhân là do Business vừa thay đổi rule (cập nhật ngưỡng score và kpi_count), và script này đã được sửa trực tiếp để chạy cho toàn bộ dữ liệu.

- **Yêu cầu:** Bạn nhận xét gì về cách viết code và quản lý logic ở đoạn script trên? Hãy đưa ra các đề xuất (suggestion) cụ thể để cải thiện khả năng bảo trì (maintainability), audit, và kiểm soát phiên bản (version control) khi Business thay đổi rule trong tương lai.